Concepts

Accounting

The accounting subsystem tracks balances, margins, and PnL for every account the platform interacts with. This guide covers the data model, the query API that strategies use, and the conventions adapter authors must follow to stay consistent across venues.

It applies equally to backtest and live trading. For backtest-specific configuration (starting balances, margin-model selection per venue), see Backtesting.

Account types

When you attach a venue to the engine for either live trading or a backtest, you pick one of three accounting modes via `account_type`:

Account type	Typical use case	What the engine locks
Cash	Spot trading (e.g., BTC/USDT, stocks)	Notional value for every position a pending order would open.
Margin	Derivatives or any product that allows leverage	Initial margin for each order plus maintenance margin for open positions.
Betting	Sports betting, bookmaking	Stake required by the venue; no leverage.

Cash accounts

Cash accounts settle trades in full; there is no leverage and therefore no concept of margin. Locked balances reflect the notional reserved for pending orders.

Margin accounts

Margin accounts support instruments that require collateral, such as futures or leveraged crypto perps. They track account balances, reserve margin for open orders and positions, and apply a configurable leverage per instrument. Margin is tracked in two scopes; see Margin scopes below.

Key terms:

- **Leverage**: amplifies exposure relative to account equity. Higher leverage raises both potential returns and risk.
- **Initial margin**: collateral reserved when an order is submitted.
- **Maintenance margin**: minimum collateral required to keep an open position.
- **Locked balance**: funds reserved as collateral, not available for new orders.

> Reduce-only orders do not contribute to `balance_locked` on cash accounts and do not add to initial margin on margin accounts, since they can only decrease exposure.

Betting accounts

Betting accounts are specialised for venues where you stake an amount to win or lose a fixed payout (prediction markets, sports books). The engine locks only the stake required by the venue; leverage and margin do not apply.

Balance model

An `AccountBalance` holds three values in the same currency:

- `total`: the venue-reported total balance figure (wallet, net liquidation, or margin balance, depending on the venue).
- `locked`: amount reserved against open orders and positions.
- `free`: amount available for new orders (`total - locked`).

The invariant `total == locked + free` must always hold at currency precision.

The Python `AccountBalance(total, locked, free)` constructor requires all three fields up front. Adapter code written in Rust has two additional derived constructors that enforce the

invariant centrally; prefer them over `AccountBalance::new` whenever the venue reports only two of the three values:

Rust helper	When to use
`AccountBalance::from_total_and_locked`	Venue reports total and locked; `free` is derived and clamped to `[0, total]`.
`AccountBalance::from_total_and_free`	Venue reports total and free; `locked` is derived and clamped.
`AccountBalance::new`	All three values are already known and consistent (tests, pass-through).

The helpers clamp the derived field to `[0, total]` when `total >= 0`, so transient overshoots from venue rounding never leave the account in a broken state.

Margin scopes

A `MarginBalance` has four fields: `initial`, `maintenance`, `currency`, and an `Optional[InstrumentId]` that selects one of two scopes.

Per-instrument scope

`MarginBalance.instrument_id` is set to a concrete instrument. Use this for:

- Isolated margin (per-position collateral), such as some OKX unified or Bybit isolated modes.
- Backtest or calculated margin, where the `AccountsManager` derives margin locally from open orders and positions per instrument.

Account-wide scope

`MarginBalance.instrument_id` is `None`. The entry is keyed by its `currency` (the collateral currency). Use this for:

- Cross-margin venues reporting a single aggregate per collateral. Examples: Binance USDT-M (USDT) and COIN-M (one per base coin), OKX, BitMEX, Hyperliquid (USDC), Bybit UNIFIED (per coin), Deribit (per currency), Kraken Futures.

Both scopes coexist on the same `MarginAccount` in separate internal stores. An `AccountState` event may carry entries in either or both scopes, and `MarginAccount.apply()` routes each entry to the correct store based on whether `instrument_id` is set.

> `MarginAccount.apply()` **replaces** both stores from the incoming event. It does not merge with prior state. Adapters that emit partial snapshots must include every live margin entry on each update or those entries will be dropped until the next full snapshot. The balances list is likewise replaced.

Strategy query API

Use the query that matches the venue's reporting shape. If a venue reports per-instrument margins, ask by `InstrumentId`. If it reports account-wide margins, ask by `Currency`.

Scope of the value you want	Use
Per-instrument margin (isolated)	`margin(id)` / `margin_init(id)` / `margin_maint(id)`
Account-wide margin for one collateral	`margin_for_currency(ccy)` / `margin_init_for_currency(ccy)` / `margin_maint_for_currency(ccy)`
Combined total across both scopes	`total_margin_init(ccy)` / `total_margin_maint(ccy)`

Point queries return `None` when the entry is absent; total queries always return a `Money` (zero for the currency if nothing matches).

> The names below are the Python / Cython API on `MarginAccount`. Rust strategies using the `nautilus-model` crate call `account_margin(&currency)`, `account_initial_margin(&currency)`, `account_maintenance_margin(&currency)`, `total_initial_margin(currency)`, and `total_maintenance_margin(currency)`: the same split by `Option<InstrumentId>`, with different method names.

Per-instrument queries (`MarginAccount`)

- `margin(instrument_id) -> MarginBalance | None`
- `margin_init(instrument_id) -> Money | None`

- `margin_maint(instrument_id) -> Money | None`
- `margins() -> dict[InstrumentId, MarginBalance]` (all per-instrument entries)
- `margins_init() -> dict[InstrumentId, Money]`
- `margins_maint() -> dict[InstrumentId, Money]`

These methods only see the per-instrument store. On a cross-margin venue they return empty dicts or `None`. Use the account-wide queries below.

Account-wide queries (`MarginAccount`)

- `margin_for_currency(currency) -> MarginBalance | None`
- `margin_init_for_currency(currency) -> Money | None`
- `margin_maint_for_currency(currency) -> Money | None`
- `account_margins() -> dict[Currency, MarginBalance]` (all account-wide entries)
- `account_margins_init() -> dict[Currency, Money]`
- `account_margins_maint() -> dict[Currency, Money]`

Totals (`MarginAccount`)

These sum across per-instrument and account-wide entries for a given currency:

- `total_margin_init(currency) -> Money`
- `total_margin_maint(currency) -> Money`

Useful when a strategy trades on a venue where both scopes may appear (for example, isolated positions alongside cross-margin collateral).

Clearing account-wide entries

- `clear_account_margin(currency)` removes the account-wide entry for a given collateral currency and triggers a balance recalculation. The counterpart for per-instrument entries is `clear_margin(instrument_id)`.

These are system methods; adapter code calls them implicitly via `MarginAccount.apply()`. Strategies should not need them directly.

Portfolio-level queries

Margin queries:

- `portfolio.margins_init(venue=..., account_id=...) -> dict[InstrumentId, Money]`
- `portfolio.margins_maint(venue=..., account_id=...) -> dict[InstrumentId, Money]`

These mirror `MarginAccount.margins_init` / `margins_maint` and return only the per-instrument entries. For account-wide data on cross-margin venues, query the account directly via `portfolio.account(venue).margin_init_for_currency(ccy)`.

PnL, exposure, mark-to-market, and equity queries all accept `venue` and an optional `account_id` to scope multi-account venues:

- `portfolio.unrealized_pnls(venue=..., account_id=...) -> dict[Currency, Money]`
- `portfolio.realized_pnls(venue=..., account_id=...) -> dict[Currency, Money]`
- `portfolio.total_pnls(venue=..., account_id=...) -> dict[Currency, Money]`
- `portfolio.net_exposures(venue=..., account_id=...) -> dict[Currency, Money]`
- `portfolio.mark_values(venue=..., account_id=...) -> dict[Currency, Money]`
- `portfolio.equity(venue=..., account_id=...) -> dict[Currency, Money]`
- `portfolio.missing_price_instruments(venue) -> list[InstrumentId]`

See the Portfolio guide for the equity formula, price fallback chain, base-currency conversion behavior, and the warn-once missing-price tracker.

Worked examples

Hyperliquid (single-collateral USDC cross margin):

```
usdc_margin = margin_account.margin_init_for_currency(USDC)
usdc_total  = margin_account.total_margin_init(USDC)
```

Bybit UNIFIED (per-coin cross margin):

```
for ccy, margin_balance in margin_account.account_margins().items():
    print(ccy, margin_balance.initial, margin_balance.maintenance)
```

dYdX v4 (USDC cross margin, aggregated per quote currency):

```
usdc_margin = margin_account.margin_init_for_currency(USDC)
```

Margin models

NautilusTrader provides flexible margin calculation models for the calculated path (backtests, and live strategies running with `calculate_account_state=True` for reconciliation). Reported margins from a venue flow straight into `_account_margins` or `_margins` without going through a model.

Overview

Different venues treat leverage differently:

- **Traditional brokers** (Interactive Brokers, TD Ameritrade): fixed margin percentages regardless of leverage.
- **Crypto exchanges** (Binance, others): leverage may reduce margin requirements.

Both built-in models compute margin as a percentage of notional using the instrument's `margin_init` and `margin_maint` fields. They differ only in whether leverage reduces the reservation. For venues with true per-contract fixed margin (CME / ICE), set `instrument.margin_init` and `margin_maint` so the percentage recovers the desired dollar amount, or implement a custom model.

Available models

`StandardMarginModel`

Uses fixed percentages without leverage division, matching traditional broker behavior.

```
# Fixed percentages - leverage ignored
margin = notional * instrument.margin_init
```

- Initial margin: `notional_value * instrument.margin_init`
- Maintenance margin: `notional_value * instrument.margin_maint`

Use cases: traditional brokers (Interactive Brokers), forex brokers with fixed margin requirements.

`LeveragedMarginModel`

Divides margin requirements by leverage.

```
# Leverage reduces margin requirements
adjusted_notional = notional / leverage
margin = adjusted_notional * instrument.margin_init
```

- Initial margin: `(notional_value / leverage) * instrument.margin_init`
- Maintenance margin: `(notional_value / leverage) * instrument.margin_maint`

Use cases: crypto exchanges that reduce margin with leverage, venues where leverage affects margin requirements.

Default behavior

`MarginAccount` uses `LeveragedMarginModel` by default. Override programmatically:

```
from nautilus_trader.backtest.models import LeveragedMarginModel
from nautilus_trader.backtest.models import StandardMarginModel
from nautilus_trader.test_kit.stubs.execution import TestExecStubs

account = TestExecStubs.margin_account()

# Traditional broker behavior
account.set_margin_model(StandardMarginModel())

# Or the leveraged model (default)
account.set_margin_model(LeveragedMarginModel())
```

Worked example: EUR/USD

- **Instrument**: EUR/USD
- **Quantity**: 100,000 EUR
- **Price**: 1.10000
- **Notional**: $110,000
- **Leverage**: 50x

- `instrument.margin_init`: 3%

Model	Calculation	Result	Percentage
Standard	$110,000 × 0.03	$3,300	3.00%
Leveraged	($110,000 ÷ 50) × 0.03	$66	0.06%

On a $10,000 account: the standard model blocks the trade; the leveraged model allows it.

Custom models

Subclass `MarginModel` and receive configuration through `MarginModelConfig`:

```
from decimal import Decimal

from nautilus_trader.backtest.config import MarginModelConfig
from nautilus_trader.backtest.models import MarginModel
from nautilus_trader.model.objects import Money


class RiskAdjustedMarginModel(MarginModel):
    def __init__(self, config: MarginModelConfig) -> None:
        self.risk_multiplier = Decimal(str(config.config.get("risk_multiplier", 1.0)))
        self.use_leverage = config.config.get("use_leverage", False)

    def calculate_margin_init(self, instrument, quantity, price, leverage, use_quote_for_inverse=
        notional = instrument.notional_value(quantity, price, use_quote_for_inverse)

        if self.use_leverage:
            adjusted = notional.as_decimal() / leverage
        else:
            adjusted = notional.as_decimal()

        margin = adjusted * instrument.margin_init * self.risk_multiplier
        return Money(margin, instrument.quote_currency)

    def calculate_margin_maint(self, instrument, side, quantity, price, leverage, use_quote_for_i
        return self.calculate_margin_init(instrument, quantity, price, leverage, use_quote_for_in
```

For backtest-wide configuration of the margin model via `BacktestVenueConfig` and `MarginModelConfig`, see the margin-models section of Backtesting.

Adapter convention

Live adapters translate venue responses into `AccountBalance` and `MarginBalance` instances. The convention that adapter authors must follow:

Building `AccountBalance`

Prefer the derived helpers so that clamping and the `total == locked + free` invariant are enforced centrally. Hand-computing three fields and passing them to `AccountBalance::new` is only appropriate for pass-through paths where all three values are already authoritative (e.g., tests).

Building `MarginBalance`

Pick the scope that matches what the venue reports:

Venue reports	Scope	Emit with
Per-instrument (isolated positions)	Per-instrument	`MarginBalance::new(initial, maint, Some(id))`
Single aggregate per collateral (cross margin)	Account-wide	`MarginBalance::new(initial, maint, None)`
Multiple aggregates, one per collateral	Account-wide	One `MarginBalance` per currency with `instrument_id=None`

Current live-adapter convention

Adapter	Scope	Collateral currencies
Binance Futures	Account-wide	USDT-M: USDT (or BNB/etc. under multi-assets mode); COIN-M: one per base coin (BTC, ETH, ...)
Bybit	Account-wide	One per coin (USDT, BTC, USDC, ...); sums position IM + order IM
Deribit	Account-wide	One per currency (BTC, ETH, USDC, ...)
Hyperliquid	Account-wide	USDC
OKX	Account-wide	USD (unified account aggregate)

Adapter	Scope	Collateral currencies
BitMEX	Account-wide	Per collateral currency (XBT, USDT, ...)
Kraken Futures	Account-wide	USD
dYdX v4	Account-wide	Computed per-position, aggregated per quote currency (USDC)
Interactive Brokers	Account-wide	Per account currency

Synthetic `ACCOUNT.{VENUE}` or `ACCOUNT-{COIN}.{VENUE}` `InstrumentId` placeholders are not used. Account-wide entries carry `instrument_id=None` and are keyed by `currency`.

Migration notes

1.226.0

`MarginBalance.instrument_id` became `Optional[InstrumentId]`, and `MarginAccount` split its internal storage into per-instrument and account-wide stores. If your strategy previously used `portfolio.margins_init(account_id=...)` to discover cross-margin balances via synthetic IDs, migrate to:

```
account = portfolio.account(venue)

# All account-wide margins for this account
account_margins = account.account_margins_init()

# Specific collateral currency
usdc_margin = account.margin_init_for_currency(USDC)

# Sum of per-instrument + account-wide for a currency
total = account.total_margin_init(USDC)
```

The per-instrument query API (`margin_init(instrument_id)`, `margins_init()`) is unchanged and now has strict per-instrument semantics.

Related guides

- Backtesting: starting balances, `MarginModelConfig`, and backtest-specific account setup.
- Portfolio: portfolio-level PnL, exposures, and currency conversion.
- Positions: position lifecycle, aggregation, and PnL.
- Adapters: requirements and best practices for adapter authors.

‹ Message Bus
Previous Page

Portfolio ›
Next Page